03 / RIESGO Y MITIGACIÓN

Evaluación de seguridad del login de práctica

Hallazgo, medida correctiva y riesgos que todavía requieren atención

Miranda Amaro Hernandez | 26 de octubre de 2026

RESUMEN

El riesgo principal era una autenticación eludible: una persona podía obtener la respuesta de acceso sin presentar la contraseña correcta. La consulta parametrizada reduce ese riesgo concreto. El ejercicio también muestra por qué una corrección de SQL injection no equivale a un sistema de autenticación listo para producción.

01 Hallazgo

La aplicación de laboratorio utiliza Express y una base SQLite temporal. El servidor recibía datos del formulario y los concatenaba en el WHERE de una consulta. Ese límite de confianza se rompía porque la entrada del cliente podía convertirse en sintaxis SQL.

Impacto observado: en la versión original, una entrada como ' OR 1=1 -- lograba ok: true y devolvía el usuario admin. La respuesta demostraba un bypass de la comprobación de contraseña dentro de esta aplicación de prueba.

02 Medida correctiva

El servidor ahora usa un SELECT con dos marcadores ? y entrega username y password como parámetros a db.get. SQLite trata la entrada como valor y no como parte del programa SQL. La ruta se mantuvo para conservar la integración con el formulario.

CONSULTA ORIGINAL / REPRESENTACIÓN DEL FALLO

```
const query = `SELECT * FROM users WHERE username = '${username}' AND password = '${password}'`;
```

03 Evidencia de funcionamiento

Se verificó la condición de seguridad junto con el uso normal. Una defensa útil debe rechazar los intentos de inyección y seguir aceptando las credenciales válidas.

04 Riesgos residuales y próximos pasos

Contraseñas: las cuentas de ejemplo se guardan en texto plano. Para un sistema real deberían almacenarse con un algoritmo de hash para contraseñas, como Argon2id o bcrypt, con parámetros adecuados.

Interfaz y abuso: el campo de contraseña utiliza type="text" y muestra lo escrito. Debe cambiarse a type="password". También conviene limitar intentos y registrar eventos sin incluir contraseñas.

Alcance: la base está en memoria y el servidor solo escucha en el equipo local. Esta práctica demuestra la mitigación de una vulnerabilidad; no implementa sesiones, control de acceso ni despliegue de producción.

CONSULTA PARAMETRIZADA / COMPARACIÓN

```
const query = "SELECT * FROM users WHERE username = ? AND password = ?";
db.get(query, [username, password], callback);
```

Caso	Resultado tras el cambio
Credencial válida: admin/admin	200 - autorizada
Credencial válida: juan/1234	200 - autorizada
Inyección: ' OR 1=1 -- / x	401 - rechazada
Inyección: admin'-- / z	401 - rechazada

Conclusión

La medida aplicada cierra el bypass de autenticación mostrado en la práctica porque conserva fija la estructura del SQL. El resultado es verificable: accesos legítimos aceptados e intentos de inyección rechazados. Para avanzar hacia una aplicación real, la siguiente prioridad sería proteger las contraseñas y reducir el abuso del formulario.

Fuente: reporte de práctica proporcionado por el usuario y verificación local del proyecto SeguridadInformatica (login/index.html y login/server-vulnerable.js).

EVIDENCIA VISUAL / MIRANDA AMARO HERNANDEZ

Capturas de la práctica

Las dos capturas muestran el mismo intento de inyección antes y después de parametrizar la consulta.

01 Antes de la corrección



El texto de inyección con contraseña «x» obtuvo ok: true y devolvió admin.

02 Después de la corrección



La consulta parametrizada rechazó el mismo intento: ok: false y «Credenciales inválidas».

Capturas del navegador proporcionadas por el usuario durante la práctica.